Programação Orientada a Objetos

Aula 03: Programação em Java

custom types, entrada e saída, *operadores e controle de fluxo*

Prof. Fabricio Santana

2025.1

fabricio.santana@idp.edu.br

www.linkedin.com/in/fabriciofsantana

O que estudamos na última aula?

http://vevox.app

ID: 170-695-337

https://idp2.vevox.com/#/present/714745

Tipos em um programa Java

- Java é uma linguagem fortemente tipada (*strongly typed*)
  - *Primitive Types*
  - *Reference Types: built-in types, custom types*

Primitive Types

- byte, short, int, long
- float, decimal
- char
- boolean

Built-in Types

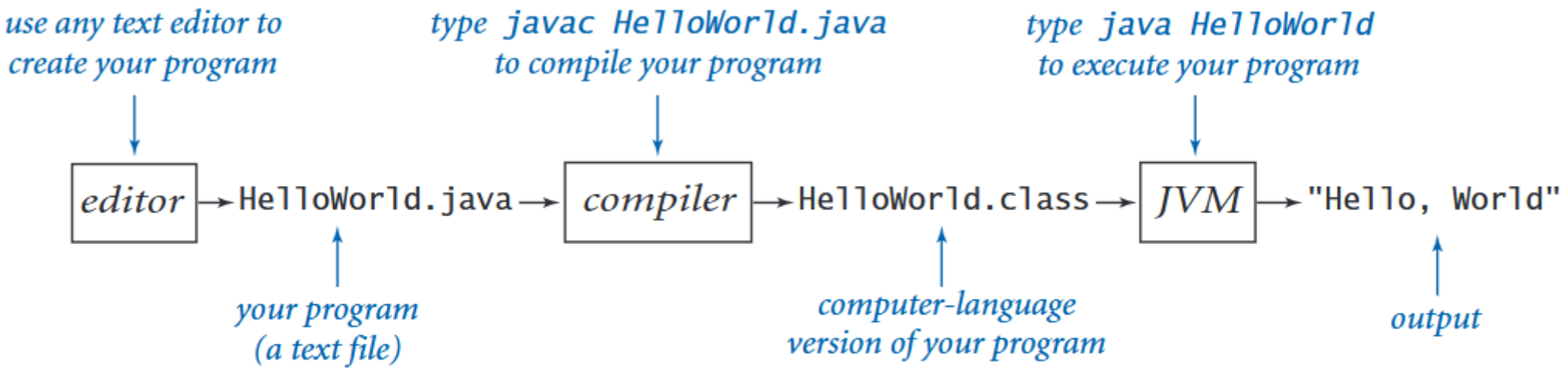
- Wrappers (Integer, ...)
- String
- ...

Custom Types

- *Cliente*
- *Conta*
- *Aluguel*
- *...*

Desenvolvendo um programa java

- Write once, run anywhere (WORA)



Developing a Java program

Fonte: SEDEGWICK, Robert; WAYNE, Kevin. Computer Science: An Interdisciplinary Approach. Boston: Addison-Wesley Professional, 2016.

Classes Java (*built-in types*)

- Tudo em java é um objeto/classe
- Java oferece uma série de classes prontas para reuso
- Algumas classes da API Java
  - java.lang.System.in: *standart input*
  - java.lang.System.out: *stantard output*
  - java.util.Scanner: lê diferente tipos de dados
  - java.lang.Math: classe com métodos para cálculos
  - java.lang.String: uma sequência de caracteres
    - “Hello, World!” → [‘H’, ‘e’ , ‘l’ , ‘l’ , ‘o’ , ‘ ’ , ‘W’ , ‘o’ , ‘r’ , ‘l’ , ‘d’, ‘!’]

Fonte: https://docs.oracle.com/en/java/javase/21/docs/api/index.html

Classes Java (*custom types*)

- public class <nome da classe> { <corpo da classe> }
  - public: modificador de acesso
  - class: palavra reservada para definição de uma classe (*custom type*)
  - <nome da classe>: identificador da classe, deve ter o mesmo nome do arquivo .java
    - <corpo da classe>
      - <variáveis de instância>: atributos da classe
      - <método>(<tipo> <nome parâmetro>)
        - <corpo do método>: coleção de *statements* que executam uma tarefa
          - <variáveis locais>
          - [return]: para métodos que retornam valor

- Como reutilizar classes java (*built-in types)* ou (*custom types)*
  - import <nome da classe>

Fonte: https://docs.oracle.com/en/java/javase/21/docs/api/index.html

Classes Java (*custom types*)

- **main**
  - método especial que é chamado pela JVM quando inicia um programa java

```java
public class HelloWorld{
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/00-hello/src

Executando o programa

- `javac -d bin src/HelloWorld.java`
  - `javac -d bin src/*.java`

- `java -cp bin/ HelloWorld`

https://docs.oracle.com/en/java/javase/21/docs/specs/man/index.html

Estruturas de controle (*control statements*)

```java
import java.util.Scanner;

public class Analysis {

    public static void main(String[] args) throws Exception {

        try(Scanner input = new Scanner(System.in)){
            int passes = 0;
            int failures = 0;
            int studentCounter = 1;

            while (studentCounter <= 10) {
                System.out.print("Enter result (1 = pass, 2 = fail): ");
                int result = input.nextInt();

                if (result == 1) {
                    passes = passes + 1;
                } else {
                    failures = failures + 1;
                }

                studentCounter = studentCounter + 1;
            }

            System.out.printf("Passed: %d%nFailed: %d%n", passes, failures);

            if (passes > 8) {
                System.out.println("Bonus to instructor!");
            }
        }
    }
}
```

- Atividade em sala, escolher uma dupla
  - Cada dupla: analisar e executar o código das classes
    - If*.java
    - Operator*.java
    - Switch*.java
    - DoWhile*.java
    - For*.java
    - Jump*.java
    - While*.java
    - Custom*.java
- Discussão em sala

Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/04-selection-statements/src
Fonte: https://github.com/fabriciosantana/poo/tree/2025.1/lectures/examples/05-iteration-statements/src

Estruturas de controle (*control statements*)

- Regra: *statements* são executados em sequência (*sequence execution*)
  - Exceção: *transfer of control (jump statements* → *break*, *continue*, *return, throw*

- Qualquer programa é escrito com apenas **3 estruturas de controle**
  - **estrutura de sequência** (*sequence statements*)
  - **estrutura de seleção** (*selection statements*)
    - ***if*** *(condition) {statements}* → *single-selection statement*
      - *operador ternário* → *... ? ... : ...*
    - ***if*** *(condition) {statements}* ***else*** *{statements}* → *double-selection statement*
    - ***switch*** *(expression) {****case*** ***value:*** *staments;* ***break*** *}* → *multiple-selection statement*
      - ***switch*** *(expression) {****case*** ***value ->*** *staments;}*
  - **estrutura de iteração** (*iteration statements*)
    - ***while*** *(condition) {statement; increment}*
    - ***do*** *{statement}* ***while*** *(condition)*
    - ***for*** *(counter initialization;condition;counter increment) {statements}*
    - *enhanced* ***for***

Estruturas de controle (*control statements*)

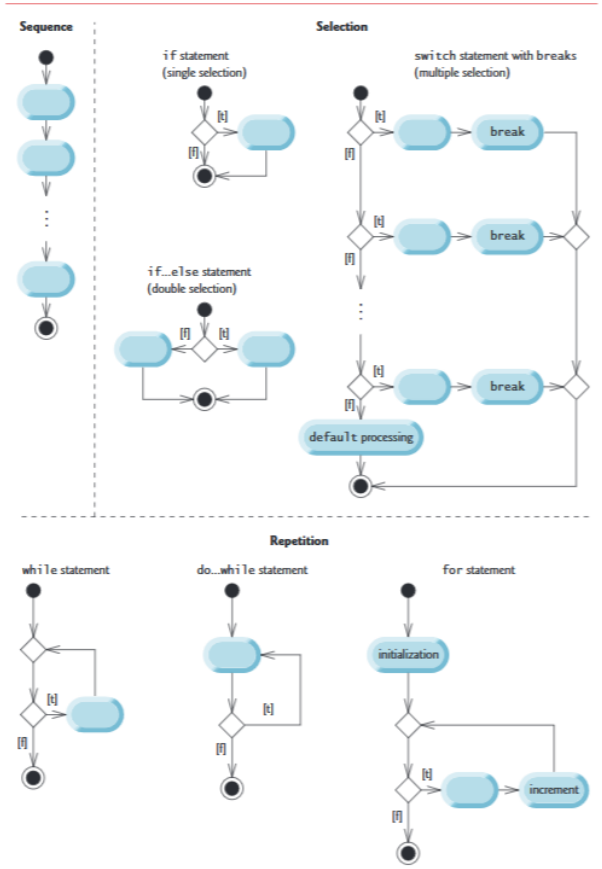


Fig. 5.21 | Java's single-entry/single-exit sequence, selection and iteration statements.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Counter-Controlled Interation

- Variável de controle (*control variable* ou *loop counter*)
- Valor inicial
- Incremento
- Condição (*loop-condition continuation*)

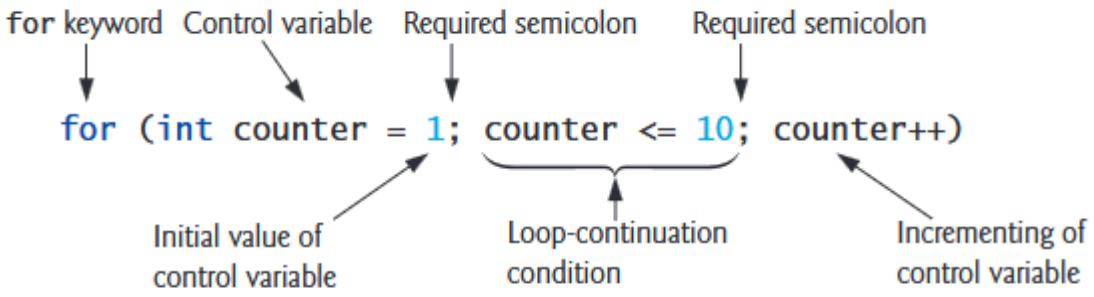


Fig. 5.3 | `for` statement header components.

Operators

- Aritmético

Operator	Result
+	Addition (also unary plus)
–	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
– =	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
– –	Decrement

- Relacional

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

- Lógico

Operator	Result
&	Logical AND
|	Logical OR
^	Logical XOR (exclusive OR)
||	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
|=	OR assignment
^=	XOR assignment

- Bitwise

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
|=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

Fonte: SCHILDT, Herbert. Java: The Complete Reference. 12. ed. New York: McGraw-Hill Education, 2021. ISBN 978-1-26-046342-2.

Compound Assignment Operator

Assignment operator	Sample expression	Explanation	Assigns
Assume: `int c = 3, d = 5, e = 4, f = 6, g = 12;`			
`+=`	`c += 7`	`c = c + 7`	10 to c
`-=`	`d -= 4`	`d = d - 4`	1 to d
`*=`	`e *= 5`	`e = e * 5`	20 to e
`/=`	`f /= 3`	`f = f / 3`	2 to f
`%=`	`g %= 9`	`g = g % 9`	3 to g

Fig. 4.13 | Arithmetic compound assignment operators.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Compound Assignment Operator

Operator	Sample expression	Explanation
++ (prefix increment)	`++a`	Increment `a` by `1`, then use the new value of `a` in the expression in which `a` resides.
++ (postfix increment)	`a++`	Use the current value of `a` in the expression in which `a` resides, then increment `a` by `1`.
-- (prefix decrement)	`--b`	Decrement `b` by `1`, then use the new value of `b` in the expression in which `b` resides.
-- (postfix decrement)	`b--`	Use the current value of `b` in the expression in which `b` resides, then decrement `b` by `1`.

Fig. 4.14 | Increment and decrement operators.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

Conclusão

- Qualquer algoritmo pode ser desenvolvido combinando:
  - **estrutura de sequência**
  - **estrutura de seleção**
    - if, if...else e switch
  - **estrutura de iteração**
    - while, do...while e for.

Fonte:DEITEL, Paul; DEITEL, Harvey. Java: How to Program, Early Objects. 11. ed. Boston: Pearson, 2017. ISBN 978-0-13-474335-6.

2ª Leitura

- 2ª Leitura: SEDGEWICK, Robert; WAYNE, Kevin. **Your first program.** In: ____. *Computer science: an interdisciplinary approach.* Boston: Addison-Wesley Professional, 2016. Cap. 1, seção 1.1. ISBN 978-0-13-407642-3.

- Prazo: 07/03/2025 às 8h15

- **Relembrando**: 1ª atividade para o 28/02/2025 até 8h15
  - Validar ambiente com Hello, World!
  - Seguir instruções gerais para entrega das atividades e a especificação das tarefas dentro de cada diretório

INSTITUTO BRASILEIRO DE ENSINO, DESENVOLVIMENTO E PESQUISA

fabricio.santana@idp.edu.br
www.linkedin.com/in/fabriciofsantana